

Control flow

COMPLETE LEARNING GUIDE

What You'll Learn

Control flow statements determine the order in which a Java program's instructions execute, and while the basic if/else and for/while constructs are near-universal across languages, Java has specific rules and modern additions — labeled break/continue, the traditional colon-based switch versus the Java 14+ arrow-based switch expression with yield — that come up constantly in both written code and whiteboard interview questions. This guide covers every branching and looping construct in Java, the difference between a switch statement (executes code, no value) and a switch expression (produces a value, exhaustive), how labeled loops let you break out of nested loops cleanly, and the classic fall-through pitfall that has caused production bugs for decades. Mastering these constructs fluently — including reading them quickly under interview time pressure — is a prerequisite for every algorithmic topic that follows in the DSA modules.

1. if / else if / else

Java evaluates conditions top to bottom, executing the first branch whose condition is true and skipping the rest — an else if chain is really a series of nested else { if (...) } blocks that Java's syntax lets you write flat. Every condition must evaluate to a boolean; unlike C, an int cannot be used directly as a condition (if (x) is a compile error unless x is a boolean), which eliminates an entire class of "assignment instead of comparison" bugs like if (x = 5) that plague C code. Braces are technically optional for single-statement bodies, but omitting them is a well-known source of bugs when a second statement is later added without noticing it falls outside the if — most style guides and linters require braces on every branch unconditionally.

2. switch: Statement vs. Expression

The traditional switch statement matches a value against several case labels and, critically, falls through to the next case unless a break is present — this default fall-through behavior is a famous source of bugs when a break is accidentally omitted. Java 14 introduced the switch expression using arrow syntax (case X ->), which does not fall through, can return a value directly, and the compiler enforces exhaustiveness on enums and sealed types. The yield keyword returns a value from a multi-statement case block inside a switch expression, playing the role that return plays inside a method.

JAVA

```
// Traditional switch statement -- fall-through risk
switch (day) {
    case 1:
    case 7:
        System.out.println("Weekend");
        break;           // omit this and execution falls into the next case
    default:
        System.out.println("Weekday");
}

// Modern switch expression (Java 14+) -- no fall-through, returns a value
String type = switch (day) {
    case 1, 7 -> "Weekend";
    case 2, 3, 4, 5, 6 -> "Weekday";
    default -> {
        yield "Unknown";    // yield returns a value from a block case
    }
};
```

3. for, while, and do-while Loops

The classic for loop packs initialization, condition, and increment into one line and is the standard choice when the number of iterations is known or index-based access is needed. while checks its condition before each iteration and may run zero times; do-while checks after, guaranteeing at least one execution — a distinction that matters for menu-driven programs or input-validation loops that must run once before checking user input. The enhanced for-each loop (for (Type item : collection)) iterates any Iterable or array without manual indexing, but it gives you no access to the current index and cannot be used to remove elements safely (that requires an explicit Iterator).

- for (int i = 0; i < n; i++) — best when you need the index or a fixed iteration count.
- while (condition) { ... } — condition checked before each iteration; may execute zero times.
- do { ... } while (condition); — condition checked after; always executes at least once.
- for (Type x : collection) — cleanest syntax for simple iteration, but no index and no safe removal.

4. break, continue, and Labeled Loops

break exits the nearest enclosing loop or switch immediately; continue skips the rest of the current iteration and jumps to the loop's next condition check. Both, by default, only affect the innermost loop they're written in — to break or continue an outer loop from inside a nested loop, Java uses labels: a named identifier placed before the outer loop's declaration, referenced by break label; or continue label;. This is one of Java's less commonly used but genuinely useful features, and interviewers sometimes test it specifically because many candidates only know unlabeled break/continue.

JAVA

```
outer:
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (grid[i][j] == target) {
            System.out.println("Found at " + i + ", " + j);
            break outer;        // exits BOTH loops immediately
        }
    }
}
```

5. The Ternary Operator

The conditional (ternary) operator `condition ? valueIfTrue : valueIfFalse` is a compact expression form of `if/else` that produces a value rather than executing statements, making it useful for inline assignment (`int max = (a > b) ? a : b;`) but a readability liability when nested or when the branches themselves are long expressions. A frequent interview gotcha involves mixing primitive types in the two branches — the ternary operator's result type follows numeric promotion rules, which can silently produce a wider or unexpected type (e.g. mixing an `int` and a `Double` branch can trigger unboxing that throws a `NullPointerException` if the `Double` is `null`).

6. Nested Loops and Complexity Awareness

Nested loops — a loop inside another loop — are ubiquitous in matrix processing, brute-force comparisons, and pattern-printing problems, and each level of nesting multiplies the iteration count: two nested loops each running n times perform n^2 total iterations, three nested loops perform n^3 . Recognizing this multiplicatively at a glance, before writing a single line of code, is essential for reasoning about a solution's time complexity during an interview — control flow structure and Big-O are directly linked, and interviewers expect you to state the complexity of nested-loop code without needing to trace through it manually.

Common Pitfalls

- Forgetting `break` in a traditional switch statement, causing execution to silently fall through into the next case's code.
- Writing `if (x)` where `x` is a non-boolean expression out of habit from C — Java rejects this at compile time, but candidates sometimes write pseudocode that assumes it works.
- Omitting braces on a single-statement `if/else` branch, then later adding a second statement that unintentionally falls outside the conditional.
- Using `continue` inside a nested loop when the intent was to skip the outer loop's iteration, without realizing a label is required to target the outer loop.
- Confusing `while` and `do-while`, especially in input-validation code that needs to run at least once before the condition can be evaluated.

- Mixing primitive and boxed types across ternary operator branches, triggering an unexpected unboxing `NullPointerException`.
- Underestimating the complexity of triple-nested loops during an interview by not immediately recognizing $O(n^3)$ growth.

Key Takeaways

- Traditional switch falls through without break; the modern arrow-based switch expression (Java 14+) does not, and can directly return a value.
- while checks before the loop body runs and may execute zero times; do-while checks after and always runs at least once.
- Labeled break/continue are the only way to control an outer loop from inside a nested inner loop.
- yield returns a value from a multi-statement block inside a switch expression, analogous to return inside a method.
- Java requires boolean conditions everywhere a condition is expected — there is no implicit int-to-boolean conversion.
- Nesting loops multiplies iteration counts — two nested loops over n is $O(n^2)$, three is $O(n^3)$; recognize this instantly for complexity analysis.

Learn More

Oracle Java Tutorials — Control Flow Statements

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html>

Oracle Java Tutorials — The switch Statement

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/switch.html>

Baeldung — Java Switch Expressions

<https://www.baeldung.com/java-switch>